Agenda

- Dziedziczenie
- Abstrakcja
- Interfejsy i polimorfizm

Dziedziczenie

Klasa modelująca kucharza

```
public class Cook {
  private String firstName;
  private String lastName;
  private int wage;

  public String name() { /* ...*/ }
  public Food cook(Food food) {
    /* ... */
  }
  public void receiveSalary() { /* ...*/ }
}
```

Klasa modelująca kelnera

```
public class Waiter {
  private String firstName;
  private String lastName;
  private int wage;

  public String name() { /* ...*/ }
  public void serve(Guest guest, Food food) {
    /* ... */
  }
  public void receiveSalary() { /* ...*/ }
}
```

Obie klasy są prawie takie same

```java
public class Cook {
  privateString firstName;
  private String lastName;
  private int wage;

  public String name() { /* ...*/ }
  public Food cook(Food food) {
    /* ... */
  }
  public void receiveSalary() { /* ...*/ }
}
```

```java
public class Waiter {
  private String firstName;
  private String lastName;
  private int wage;

  public String name() { /* ...*/ }
  public void serve(Guest guest, Food food) {
    /* ... */
  }
  public void receiveSalary() { /* ...*/ }
}
```

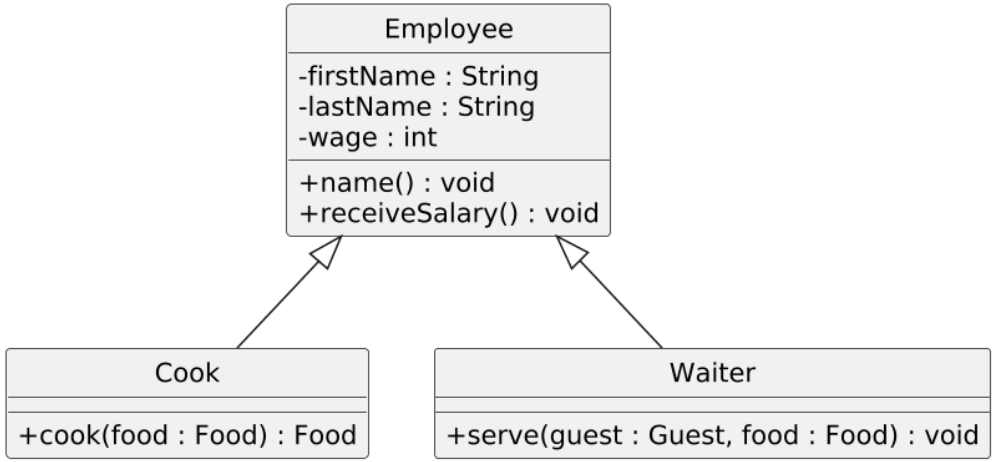
Pracownik – część wspólna obu klas

```
public class Employee {
  private String firstName;
  private String lastName;
  private int wage;

  public String name() { /* ...*/ }



  public void receiveSalary() { /* ...*/ }
}
```

Dziedziczenie

```
public class Employee {
  /* ... */
}

public class Cook extends Employee {
  public Food cook(Food food) {
    /* ... */
  }
}
public class Waiter extends Employee {
  public void serve(Guest guest, Food food) {
    /* ... */
  }
}
```

Dziedziczenie




```java
public class Employee {
  /* ... */
}

public class Cook extends Employee {
  public Food cook(Food food) {
    /* ... */
  }
}
public class Waiter extends Employee {
  public void serve(Guest guest, Food food) {
    /* ... */
  }
}
```

Klasa modelująca gościa

```
public class Guest {
  private String firstName;
  private String lastName;


  public String name() { /* ... */ }

  public void eat(Food food) { /* ... */ }
  public void pay() { /* ... */ }

}
```

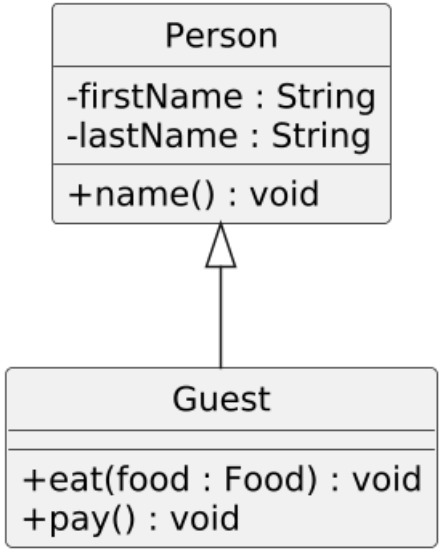
Klasa modelująca gościa

```
public class Guest {
  private String firstName;
  private String lastName;
  private int wage;

  public String name() { /* ... */ }

  public void eat(Food food) { /* ... */ }
  public void pay() { /* ... */ }
  public void receiveSalary() { /* ...*/ }
}
```

Uogólnienie do klasy osoby

```
public class Person {
  private String firstName;
  private String lastName;
  public String name() { /* ... */ }
}

public class Guest extends Person {
  public void eat(Food food) { /* ... */ }
  public void pay() { /* ... */ }
}
```

Uogólnienie do klasy osoby



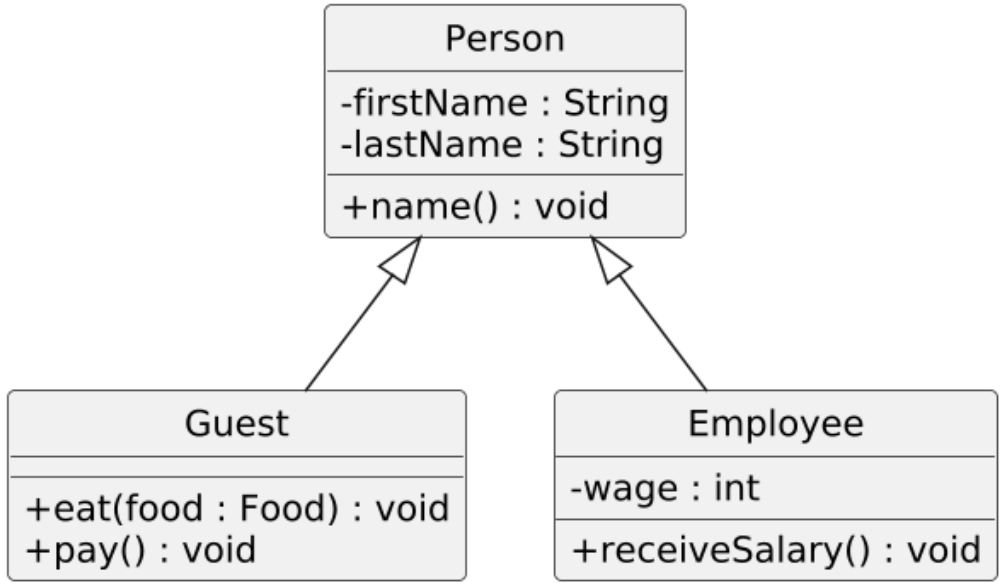

```
public class Person {
  private String firstName;
  private String lastName;
  public String name() { /* ... */ }
}

public class Guest extends Person {
  public void eat(Food food) { /* ... */ }
  public void pay() { /* ... */ }
}
```

Część wspólna pracownika i osoby

```java
public class Employee {
private String firstName;
private String lastName;
private int wage;

public String name() { /* ...*/ }



public void receiveSalary() { /* ...*/ }
}
```

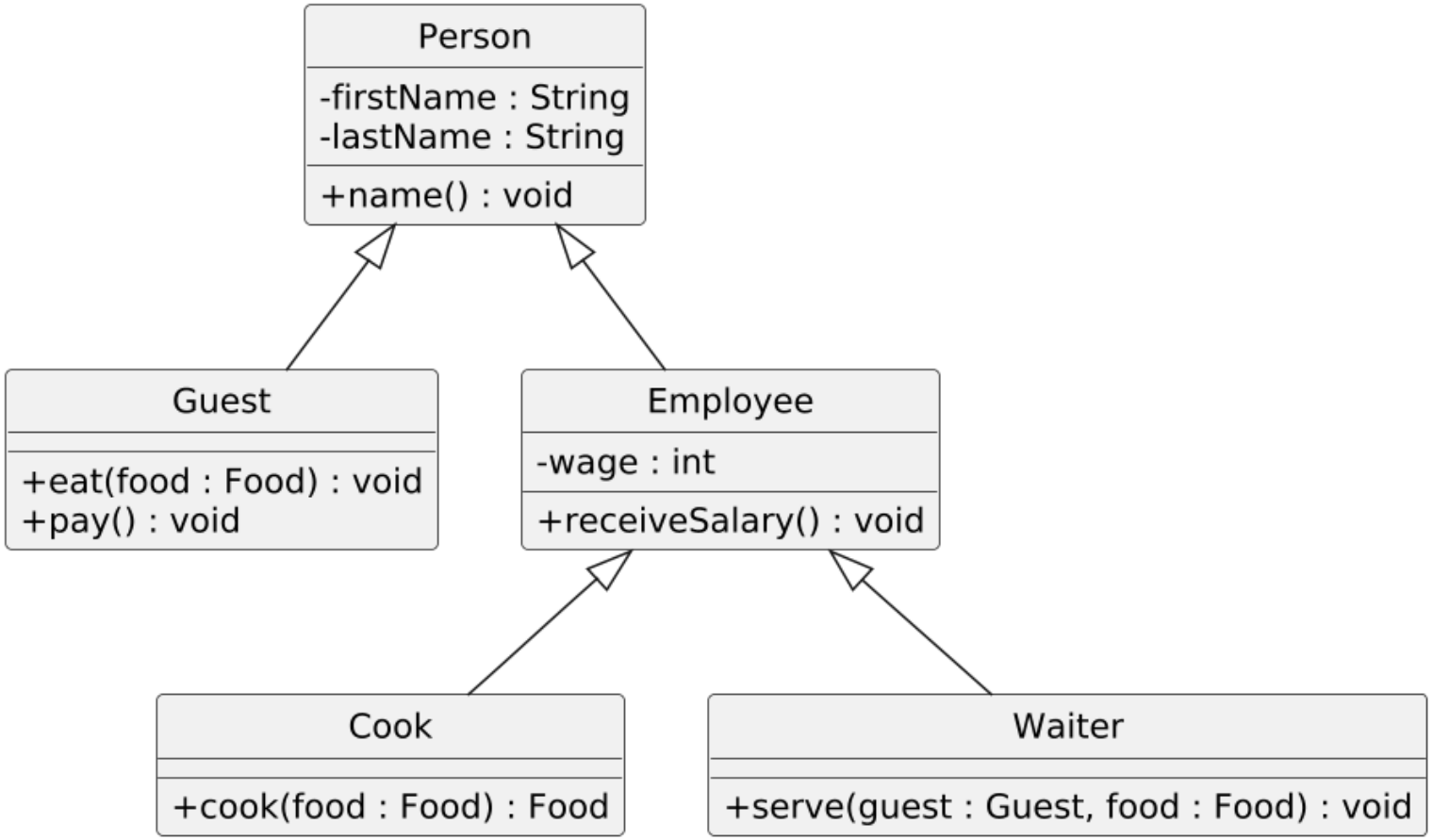
```java
public class Person {
  private String firstName;
  private String lastName;


  public String name() { /* ... */ }

}
```

Dziedziczenie pracownika po osobie




```
public class Person {
  private String firstName;
  private String lastName;
  public String name() { /* ... */ }
}

public class Employee extends Person {
  private int wage;
  public void receiveSalary() { /* ...*/ }
}

public class Guest extends Person {
  public void eat(Food food) { /* ... */ }
  public void pay() { /* ... */ }
}
```

Pełna hierarchia klas



Klasa nadrzędna definiuje metodę

```
public class Person {
  private String firstName;
  private String lastName;
  public String name() {
    return String.format("%s %s", firstName, lastName);
  }
  public void setName(String fn, String ln) {
    firstName = fn; lastName = ln;
  }
}

Person p = new Person();
p.setName("Jan", "Kowalski");
System.out.println(p.name());
// Jan Kowalski
```

Klasy podrzędne ją dziedziczą

```
public class Guest extends Person { /* ... */ }
public class Cook extends Employee { /* ... */ }

Guest guest = new Guest();
guest.setName("Anna", "Kowalska");
System.out.println(guest.name());
// Anna Kowalska

Cook cook = new Cook();
cook.setName("Piotr", "Nowak");
System.out.println(cook.name());
// Piotr Nowak
```

Klasy podrzędne mogą ją przysłonić

```
public class Person {
  /* ... */
  public String name() {
    return String.format("%s %s", firstName, lastName);
  }
}

public class Guest extends Person {
  /* ... */
  @Override
  public String name() {
    return "a guest";
  }
}
```

Klasy podrzędne mogą ją przysłonić

```java
Guest guest = new Guest();
guest.setName("Anna", "Kowalska");
System.out.println(guest.name());
// anonymous guest

Cook cook = new Cook();
cook.setName("Piotr", "Nowak");
System.out.println(cook.name());
// Piotr Nowak
```

```java
public class Person {
  /* ... */
  public String name() {
    return String.format("%s %s", firstName, lastName);
  }
}

public class Guest extends Person {
  /* ... */
  @Override
  public String name() {
    return "anonymous guest";
  }
}
```

Przysłoniętą metodę można wywołać

```
public class Person {
  /* ... */
  public String name() {
    return String.format("%s %s", firstName, lastName);
  }
}

public class Guest extends Person {
  /* ... */
  boolean anonymous;
  @Override
  public String name() {
    if (anonymous) return "a guest";
    else return super.name();
  }
}
```

Można zablokować przysłanianie

```
public class Person {
  /* ... */
  public final String name() {
    return String.format("%s %s", firstName, lastName);
  }
}

public class Guest extends Person {
  /* ... */
  boolean anonymous;
  @Override
  public String name() {
    if (anonymous) return "a guest";
    else return super.name();
  }
}
```

Dostęp do pola klasy nadrzędnej

```
public class Person {
  private String firstName;
  private String lastName;
  public final String name() {
    return String.format("%s %s", firstName, lastName);
  }
}

public class Waiter extends Employee {
  /* ... */
  public String name() {
    return firstName;
  }
}
```

`private` blokuje dostęp w klasach podrzędnych

```
public class Person {
  private String firstName;
  private String lastName;
  public String name() {
    return String.format("%s %s", firstName, lastName);
  }
}

public class Waiter extends Employee {
  /* ... */
  public String name() {
    return firstName;
  }
}
```

protected umożliwa ten dostęp

```
public class Person {
  protected String firstName;
  private String lastName;
  public String name() {
    return String.format("%s %s", firstName, lastName);
  }
}

public class Waiter extends Employee {
  /* ... */
  public String name() {
    return firstName;
  }
}
```

Enkapsulacja przy dziedziczeniu

	public	protected	private
Ta sama klasa	✓	✓	✓
Klasa dziedzicząca	✓	✓	X
Poza klasą	✓	X	X

```
public class Person {
  protected String firstName;
  private String lastName;
  public String name() {
    return String.format("%s %s", firstName, lastName);
  }
}

public class Waiter extends Employee {
  /* ... */
  public String name() {
    return firstName;
  }
}
```

Konstruktor klasy nadrzędnej...

```
public class Person {
  protected String firstName;
  private String lastName;

  public Person(String fn, String ln) {
    firstName = fn; lastName = ln;
  }
  public final String name() {
    return String.format("%s %s", firstName, lastName);
  }
}

Person person = new Person("Anna", "Kowalska");
System.out.println(person.name()); // Anna Kowalska
```

...nie jest dziedziczony...

```java
public class Guest extends Person {

  public void eat(Food food) { /* ... */ }
  public void pay() { /* ... */ }
}

Guest guest = new Guest("Anna", "Kowalska");
System.out.println(guest.name());
```

```java
public class Person {
  protected String firstName;
  private String lastName;

  public Person(String fn, String ln) {
    firstName = fn; lastName = ln;
  }
  public final String name() {
    return String.format("%s %s", firstName, lastName);
  }
}

Person person = new Person("Anna", "Kowalska");
System.out.println(person.name()); // Anna Kowalska
```

...i wymaga definicji na każdym poziomie

```java
public class Guest extends Person {



  public Guest(String fn, String ln) {

    /* ... */
  }

    public void eat(Food food) { /* ... */ }
    public void pay() { /* ... */ }
  }

  Guest guest = new Guest("Anna", "Kowalska");
  System.out.println(guest.name());
```

```java
public class Person {
  protected String firstName;
  private String lastName;

  public Person(String fn, String ln) {
    firstName = fn; lastName = ln;
  }
  public final String name() {
    return String.format("%s %s", firstName, lastName);
  }
}

Person person = new Person("Anna", "Kowalska");
System.out.println(person.name()); // Anna Kowalska
```

Enkapsulacja dotyczy też konstruktora

```
public class Guest extends Person {


public Guest(String fn, String ln) {

  firstName = fn; lastName = ln;
}

  public void eat(Food food) { /* ... */ }
  public void pay() { /* ... */ }
}

Guest guest = new Guest("Anna", "Kowalska");
System.out.println(guest.name());
```

```
public class Person {
  protected String firstName;
  private String lastName;

  public Person() {
  }
  public Person(String fn, String ln) {
    firstName = fn; lastName = ln;
  }

  public final String name() {
    return String.format("%s %s", firstName, lastName);
  }
}
```

Niejawne wywołane super()

```java
public class Guest extends Person {


public Guest(String fn, String ln) {
  super();
  firstName = fn;
}

  public void eat(Food food) { /* ... */ }
  public void pay() { /* ... */ }
}

Guest guest = new Guest("Anna", "Kowalska");
System.out.println(guest.name());
```

```java
public class Person {
  protected String firstName;
  private String lastName;

  public Person() {
  }
  public Person(String fn, String ln) {
    firstName = fn; lastName = ln;
  }

  public final String name() {
    return String.format("%s %s", firstName, lastName);
  }
}
```

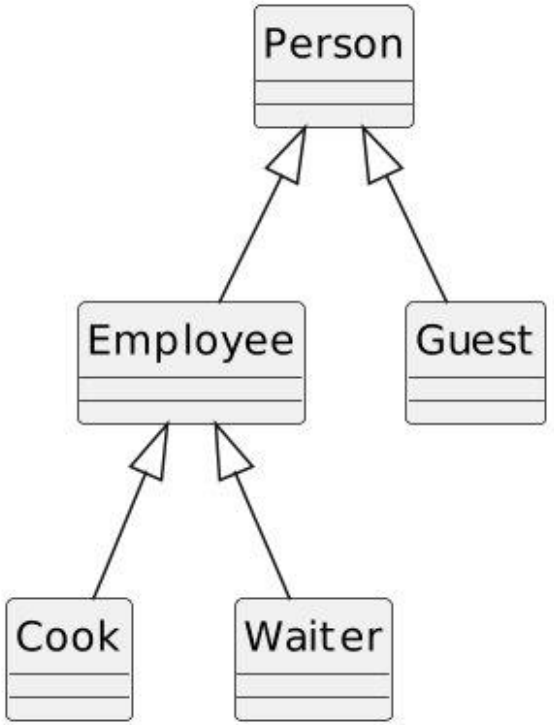
Wywołanie konstruktora klasy nadrzędnej

```java
public class Guest extends Person {


public Guest(String fn, String ln) {
  super(fn, ln);

}

  public void eat(Food food) { /* ... */ }
  public void pay() { /* ... */ }
}

Guest guest = new Guest("Anna", "Kowalska");
System.out.println(guest.name());
```

```java
public class Person {
  protected String firstName;
  private String lastName;



  public Person(String fn, String ln) {
    firstName = fn; lastName = ln;
  }

  public final String name() {
    return String.format("%s %s", firstName, lastName);
  }
}
```

Łańcuchy konstruktorów

```
public Person(String firstName, String lastName) {
  this.firstName = firstName; this.lastName = lastName;
}
public Guest(String firstName, String lastName) {
  super(firstName, lastName);
}
public Employee(String firstName, String lastName, int wage) {
  super(firstName, lastName);
  this.wage = wage;
}
public Cook(String firstName, String lastName, int wage) {
  super(firstName, lastName, wage);
}
public Waiter(String firstName, String lastName, int wage) {
  super(firstName, lastName, wage);
}
```



Abstrakcja

Małe zróżnicowanie w ramach klasy

```
public class Food {
   private final String name;
   private int weight;
   private int price;
   public Food(String name, int weight, int price) {
     this.name = name;
     this.weight = weight;
     this.price = price;
   }
}
Food potatoes = new Food("Potato", 100, 1_99);
Food tomatoes = new Food("Tomato", 80, 7_49);
Food carrots  = new Food("Carrot", 90, 3_29);
Food lettuces = new Food("Lettuce", 50, 4_99);
```

Nowy model dopasowany do kontekstu

```
public class Dish {
  public final String name;
  public Dish(String name) { this.name = name; }
}

public class Guest extends Person {
  public Guest(String firstName, String lastName) {
    super(firstName, lastName);
  }

  public void eat(Dish dish) {
    System.out.printf("%s eats %s.", name(), dish.name);
  }
}
```

Nowy model dopasowany do kontekstu

```java
Guest guest1 = new Guest("Jan", "Kowalski");
Guest guest2 = new Guest("Anna", "Kowalska");

Dish chicken = new Dish("fried chicken");
Dish salad   = new Dish("cucumber salad");

guest1.eat(chicken);
// Jan Kowalski eats fried chicken.

guest2.eat(salad);
// Anna Kowalska eats cucumber salad.
```

```java
public class Dish {
  public final String name;
  public Dish(String name) { this.name = name; }
}

public class Guest extends Person {
  public Guest(String firstName, String lastName) {
    super(firstName, lastName);
  }

  public void eat(Dish dish) {
    System.out.printf("%s eats %s.", name(), dish.name);
  }
}
```

Zbyt ubogi model dla zadanego kontekstu

```java
public class Dish {
  public final String name;
  public Dish(String name) { this.name = name; }
}

public class Cook extends Employee {
  /* ... */

  public Dish cook(Dish dish) {
    System.out.printf("%s cooks %s.", name(), dish.name);
    /* ... */
    return dish;
  }
}
```

Zła praktyka – podejście nieobiektowe

```
public class Cook extends Employee {
  /* ... */
  private Fryer fryer;


  public Dish cook(Dish dish) {
    if(dish.name.equals("fried chicken"))
      dish = prepareFriedChicken(dish);
    else if (dish.name.equals("cucumber salad"))
      dish = prepareCucumberSalad(dish);
    else
      { /* ... */ }
    return dish;
  }
```

```
  public Dish prepareCucumberSalad(Dish dish) {
    dish.addIngredient(new Food("tomato"));
    dish.addIngredient(new Food("cucumber"));
    dish.addSpices(new Food("salt");
    dish.addSpices(new Food("pepper");
    dish.addDressing(new Food("olive oil"));
    dish.mix();
    return dish
  }
}
```

Zła praktyka – podejście nieobiektowe

```java
public class Cook extends Employee {
  /* ... */
  private Fryer fryer;


  public Dish cook(Dish dish) {
    if(dish.name.equals("fried chicken"))
      dish = prepareFriedChicken(dish);
    else if (dish.name.equals("cucumber salad"))
      dish = prepareCucumberSalad(dish);
    else
      { /* ... */ }
    return dish;
  }
```

```java
  public Dish prepareFriedChicken(Dish dish) {
    dish.addIngredient(new Food("chicken"));
    dish.addSpices(new Food("salt"));
    dish.addSpices(new Food("pepper"));
    dish.season(60);
    dish.coat(new Food("flour"));
    dish.coat(new Food("egg"));
    fryer.setTemperature(180);
    fryer.heatOil();
    fryer.setTime(10);
    fryer.fry(dish);
    dish.drainOil();
    return dish;
  }
}
```

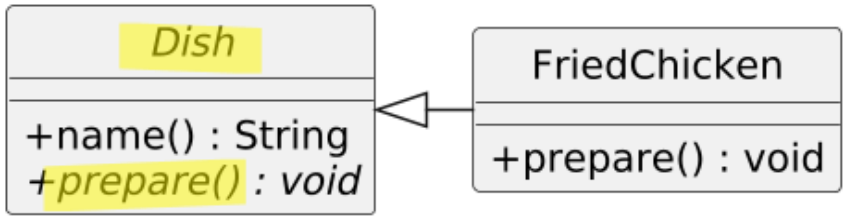
Wydzielenie osobnych klas

```java
public class Dish {
  public final String name;
  public Dish(String name) { this.name = name; }



  public void prepare() {
    System.out.println("don't know how");
  }
}
```

```java
public class FriedChicken extends Dish {
  private Fryer fryer;
  public FriedChicken() { super("fried chicken"); }
  /* ... */
  public void setFryer(Fryer fryer) { this.fryer = fryer; }

  public void prepare() {
    addIngredient(new Food("chicken"));
    addSpices(new Food("salt"));
    /* ... */
    fryer.setTemperature(180);
    fryer.setTime(10);
    fryer.fry(this);
  }
}
```

Klasa abstrakcyjna

```java
public abstract class Dish {
  public final String name;
  public Dish(String name) { this.name = name; }



  public abstract void prepare();
}
```




```java
public class FriedChicken extends Dish {
  private Fryer fryer;
  public FriedChicken() { super("fried chicken"); }
  /* ... */
  public void setFryer(Fryer fryer) { this.fryer = fryer; }

  public void prepare() {
    addIngredient(new Food("chicken"));
    addSpices(new Food("salt"));
    /* ... */
    fryer.setTemperature(180);
    fryer.setTime(10);
    fryer.fry(this);
  }
}
```

Nie istnieją obiekty klas abstrakcyjnych

```java
public abstract class Dish {
  public final String name;
  public Dish(String name) { this.name = name; }



  public abstract void prepare();
}
```

```java
Dish salad = new Dish("Beetroot salad");
```

```java
FriedChicken chicken = new FriedChicken();
```

```java
public class FriedChicken extends Dish {
  private Fryer fryer;
  public FriedChicken() { super("fried chicken"); }
  /* ... */
  public void setFryer(Fryer fryer) { this.fryer = fryer; }

  public void prepare() {
    addIngredient(new Food("chicken"));
    addSpices(new Food("salt"));
    /* ... */
    fryer.setTemperature(180);
    fryer.setTime(10);
    fryer.fry(this);
  }
}
```

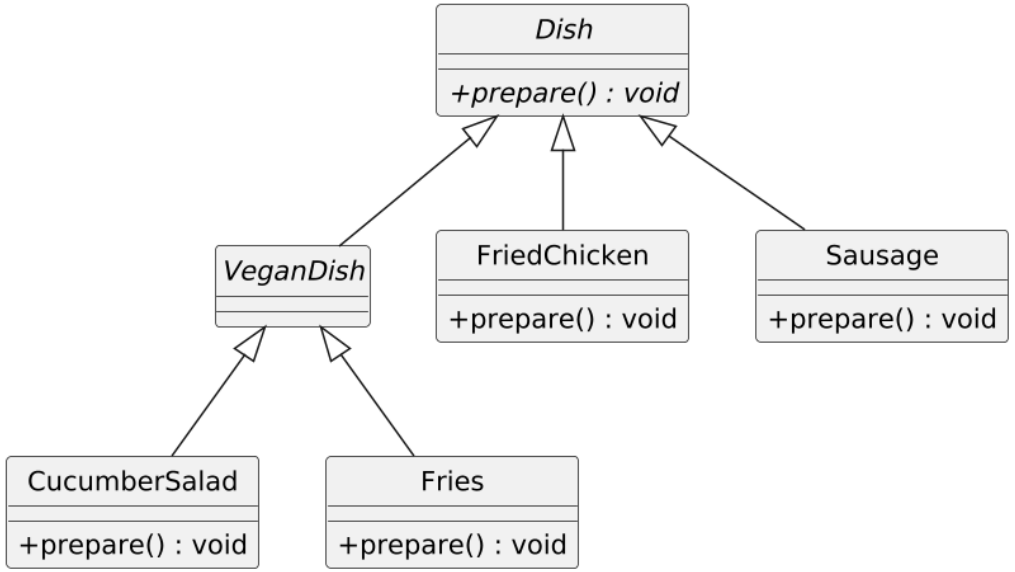
Implementacja pochodzi z klasy obiektu

```
public class Cook extends Employee {
  /* ... */

  public Dish cook(Dish dish) {
    dish.prepare();
    return dish;
  }
}

Cook cook = new Cook("Piotr", "Nowak");
FriedChiken chicken = new FriedChicken();
Dish dish = cook.cook(chicken);
```

Zmiana klasy zmienia implementację metody

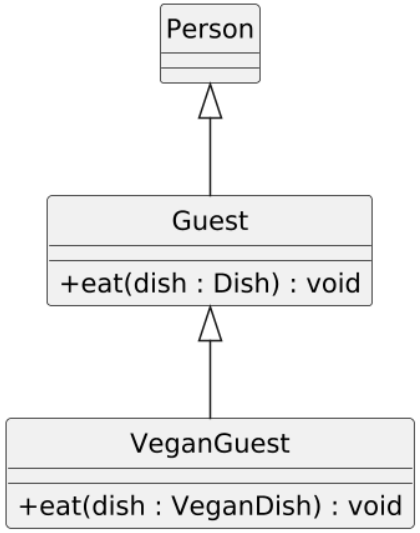
```
public class Cook extends Employee {
  /* ... */

  public Dish cook(Dish dish) {
    dish.prepare();
    return dish;
  }
}

Cook cook = new Cook("Piotr", "Nowak");
CucumberSalad salad = new CucumberSalad();
Dish dish = cook.cook(salad);
```

Metoda abstrakcyjna w hierarchii klas




```
public abstract class Dish {
  /* ... */
  public abstract void prepare();
}
public abstract class VeganDish extends Dish {
  /* ... */
}
public class FriedChicken extends Dish {
  /* ... */
  public void prepare() { /* ... */ }
}
public class CucumberSalad extends VeganFood {
  /* ... */
  public void prepare() { /* ... */ }
}
```

Obiekty powiązanych klas jako argumenty




```java
public class Guest extends Person {
  /* ... */
  public void eat(Dish dish) {
    System.out.printf("%s eats %s.\n", name(), dish.name);
  }
}

public class VeganGuest extends Guest {
  /* ... */
  public void eat(VeganDish dish) {
    super.eat(dish);
    System.out.println("By the way, I'm vegan!");
  }
}
```

Pełna zgodność typów

```java
Guest guest = new Guest("Jan", "Kowalski");
FriedChicken chicken = new FriedChicken();
guest.eat(chicken);
// Jan Kowalski eats fried chicken.


VeganGuest vegan = new VeganGuest("Anna", "Kowalska");
CucumberSalad salad = new CucumberSalad();
vegan.eat(salad);
// Anna Kowalska eats cucumber salad.
// By the way, I'm vegan!
```

```java
public class Guest extends Person {
  /* ... */
  public void eat(Dish dish) {
    System.out.printf("%s eats %s.\n", name(), dish.name);
  }
}

public class VeganGuest extends Guest {
  /* ... */
  public void eat(VeganDish dish) {
    super.eat(dish);
    System.out.println("By the way, I'm vegan!");
  }
}
```

Argument polimorficzny

```java
Guest guest = new Guest("Jan", "Kowalski");
Fries chicken = new Fries(); // VeganFood
guest.eat(fries);
// Jan Kowalski eats fries.
```

```java
public class Guest extends Person {
  /* ... */
  public void eat(Dish dish) {
    System.out.printf("%s eats %s.\n", name(), dish.name);
  }
}

public class VeganGuest extends Guest {
  /* ... */
  public void eat(VeganDish dish) {
    super.eat(dish);
    System.out.println("By the way, I'm vegan!");
  }
}
```

Co się wydarzy?

```java
Guest guest = new Guest("Jan", "Kowalski");
Fries chicken = new Fries(); // VeganFood
guest.eat(fries);
// Jan Kowalski eats fries.


VeganGuest vegan = new VeganGuest("Anna", "Kowalska");
Sausage sausage = new Sausage(); // Food
vegan.eat(ribs);
// ???
```

```java
public class Guest extends Person {
  /* ... */
  public void eat(Dish dish) {
    System.out.printf("%s eats %s.\n", name(), dish.name);
  }
}

public class VeganGuest extends Guest {
  /* ... */
  public void eat(VeganDish dish) {
    super.eat(dish);
    System.out.println("By the way, I'm vegan!");
  }
}
```

Smacznego!

```
Guest guest = new Guest("Jan", "Kowalski");
Fries chicken = new Fries(); // VeganFood
guest.eat(fries);
// Jan Kowalski eats fries.


VeganGuest vegan = new VeganGuest("Anna", "Kowalska");
Sausage sausage = new Sausage(); // Food
vegan.eat(sausage);
// Anna Kowalska eats sausage.
```

Wywołanie metody klasy nadrzędnej

```java
Guest guest = new Guest("Jan", "Kowalski");
Fries chicken = new Fries(); // VeganFood
guest.eat(fries);
// Jan Kowalski eats fries.


VeganGuest vegan = new VeganGuest("Anna", "Kowalska");
Sausage sausage = new Sausage(); // Food
vegan.eat(sausage);
// Anna Kowalska eats sausage.
```

```java
public class Guest extends Person {
  /* ... */
  public void eat(Dish dish) {
    System.out.printf("%s eats %s.\n", name(), dish.name);
  }
}

public class VeganGuest extends Guest {
  /* ... */
  public void eat(VeganDish dish) {
    super.eat(dish);
    System.out.println("By the way, I'm vegan!");
  }
}
```

Metody mają niezgodny nagłówek

```java
public class Guest extends Person {
  /* ... */


  public void eat(Dish dish) {
    System.out.printf("%s eats %s.\n",
                      name(), dish.name);
  }
}
```

```java
public class VeganGuest extends Guest {
  /* ... */

  @Override
  public void eat(VeganDish dish) {
    super.eat(dish);
    System.out.println("By the way, I'm vegan!");
  }
}
```

Zgodność nagłówka pozwala nadpisać metodę

```java
public class Guest extends Person {
  /* ... */

  public void eat(Dish dish) {
    System.out.printf("%s eats %s.\n",
                      name(), dish.name);
  }
}
```

```java
public class VeganGuest extends Guest {
  /* ... */
  @Override
  public void eat(Dish dish) {
    System.out.printf(
        "%s refuses to eat non-vegan dish.\n", name());
  }

  public void eat(VeganDish dish) {
    super.eat(dish);
    System.out.println("By the way, I'm vegan!");
  }
}
```

Zgodność nagłówka pozwala nadpisać metodę

```java
Guest guest = new Guest("Jan", "Kowalski");
Fries chicken = new Fries(); // VeganFood
guest.eat(fries);
// Jan Kowalski eats fries.


VeganGuest vegan = new VeganGuest("Anna", "Kowalska");
Sausage sausage = new Sausage(); // Food
vegan.eat(sausage);
// Anna Kowalska refuses to eat non-vegan dish.
```

```java
public class VeganGuest extends Guest {
  /* ... */
  @Override
  public void eat(Dish dish) {
    System.out.printf(
        "%s refuses to eat non-vegan dish.\n", name());
  }

  public void eat(VeganDish dish) {
    super.eat(dish);
    System.out.println("By the way, I'm vegan!");
  }
}
```